Measured resource elasticity (Tier 1, items 17 and 18)

Per query, the log-log slope of runtime against each resource, measured on the local SF1 instance. `e_dop` uses the **measured** degree of parallelism rather than the requested setting, because the planner frequently declines to go parallel at all.

Signs expected: `e_dop` negative (more cores, faster), `e_wm` negative (more memory, faster), `e_bw` positive (more bandwidth contention, slower).

query	hand label	e_dop	e_wm	e_bw	max DOP	base ms
q1	compute	-0.472	−0.053	+0.327	5	2746
q2	mixed	+0.052	−0.096	+0.519	3	254
q3	mixed	-1.257	+0.291	+0.488	4	1850
q4	io	-0.156	+0.405	+0.468	4	1209
q5	mixed	-0.657	−0.131	+1.145	3	358
q6	bandwidth	-0.647	−0.401	+1.184	5	262
q7	mixed	+0.401	+0.332	+1.201	3	2051
q8	mixed	-0.535	−0.003	+1.604	3	159
q9	mixed	-0.197	−0.009	+0.925	4	1516
q10	mixed	-0.245	−0.169	+0.586	4	317
q11	mixed	+nan	−0.030	+0.641	1	50
q12	compute	+0.223	−0.085	−0.274	5	2001
q13	bandwidth	+0.292	−0.087	+0.809	3	1302
q14	bandwidth	+0.182	−0.127	+0.505	5	1272
q16	compute	-0.633	−0.036	+0.857	5	288
q17	io	-0.021	+0.006	+0.556	3	799
q18	mixed	+0.617	+0.193	+0.135	4	3801
q19	bandwidth	-0.826	+0.049	+0.607	3	48
q20	io	+nan	+0.077	+0.383	1	719
q21	mixed	-0.735	+0.465	+1.367	4	1599
q22	io	-0.513	−0.020	+0.909	3	41

Do the measured elasticities agree with the hand labels?

Mean measured sensitivity by hand-assigned class:

hand label	compute sensitivity	bandwidth sensitivity	memcap sensitivity
bandwidth	0.368	0.776	0.154
compute	0.368	0.395	0.058
io	0.172	0.579	0.005
mixed	0.363	0.861	0.044

The diagonal does not hold.

- Compute sensitivity is highest for **compute**-labelled queries, and `compute`-labelled queries score 0.368 — identical to the top class. `compute` and `bandwidth` labelled queries have essentially the *same* measured compute sensitivity, so the label does not discriminate it at all.
- Bandwidth sensitivity is highest for **mixed**-labelled queries (0.861), not for `bandwidth`-labelled ones (0.776).
- **6 of 21 queries have a positive** `e_dop` — they run *slower* with more cores — and **1 has a negative** `e_bw`, running faster under bandwidth contention. Those are physically impossible as steady-state properties and mark queries where the effect is below the noise of a two-rep measurement.
- `q11`, `q20` show **DOP = 1**: the planner never chose a parallel plan for them, so core elasticity is undefined rather than zero. These are exactly the two queries `fix3_parallelism_mechanism.py` independently identified as plan-serial, which is a useful cross-check that the rig is measuring what it claims to.

This is measured evidence for what the classifier ablation already implied: **the four-class bottleneck taxonomy does not describe a property these queries actually have.** It was never a labelling accuracy problem.

Elasticity weights in the analytical formula

Leave-one-machine-out, multi-core compute signal. `elasticity` uses `w_bandwidth/r_bw + w_compute/r_cmp` with the measured weights; the others are as in `tier1_ablation.md`.

weighting	MAPE %	MdAPE %	q-error (median)
naive_linear	23.05	19.37	1.220
analytical	21.61	18.14	1.198
uniform	21.19	16.96	1.186
elasticity	21.76	18.08	1.203
cmp_only	20.06	15.87	1.170

Item 18's hypothesis is not supported. Measured elasticities are a better *description* of these queries than the hand labels, but as weights in this formula they do not help: `elasticity` lands at 21.76%, behind both the hand-label-driven `analytical` (21.61%) and a flat 50/50 `uniform` split (21.19%).

The ranking is the whole story: `cmp_only` (20.06%) < `uniform` (21.19%) < `analytical` (21.61%) < `elasticity` (21.76%) < `naive_linear` (23.05%). Accuracy rises monotonically as weight shifts from the bandwidth ratio onto the compute ratio, and peaks when *all* of it is on compute. No weighting scheme is the answer, because weighting is the wrong lever — the formula's second term is simply better than its first.

Paired cluster bootstrap against the bandwidth-ratio baseline:

comparison	point	95% CI	verdict
naive linear minus analytical	+1.44pp	[0.32, 2.56]	**SIGNIFICANT**
naive linear minus elasticity	+1.29pp	[0.61, 2.13]	**SIGNIFICANT**
naive linear minus cmp only	+2.99pp	[-0.09, 6.07]	not significant

Caveats that travel with every number above

- **Measured on one machine (Apple M1, arm64, 8 cores, 8 GB), not on the five EC2 instances**, which are terminated. Elasticity is being treated as a property of the query, which is the same assumption the buffer correction already relies on, but it is an assumption.
- **No CPU affinity control.** macOS has no taskset/cgroups, so the bandwidth antagonist also steals compute. `e_bw` must be read as sensitivity to combined bandwidth-and-core contention. The `e_dop` axis varies cores with no antagonist, which is what allows them to be partly separated.
- `e_bw` **is a TWO-POINT slope** (0 and 4 antagonist threads). The 8-thread level was collected and then discarded: on an 8-core machine it starves PostgreSQL of CPU rather than contending for bandwidth (q1 ran 526,827 ms against a ~4,000 ms baseline, a 130x slowdown). Without affinity control there is no way to raise bandwidth pressure past the point where core starvation dominates, so this axis is the weakest of the three and `e_bw` should be treated as indicative only.
- `shared_buffers` **was never swept**, because that needs a server restart and this PostgreSQL instance hosts other databases. Memory sensitivity is approached through `work_mem` instead, which captures spill behaviour but not buffer-cache residency.
- **q16 and q20 are APPROXIMATE reconstructions** (see `scripts/collection/queries/README.md`); their elasticities describe a query slightly more or less selective than the one in the dataset.